

DEVOPS MASTER COURSE

Volume 6E

AWS Enterprise DevOps, Observability, Security & Governance

Chapters 41-55

CLOUDFORMATION | CDK | CI/CD | CLOUDWATCH | SECURITY | GOVERNANCE | RESILIENCE |
FINOPS

Production automation, observability, compliance, threat detection, multi-account governance,
labs, troubleshooting, and enterprise capstone

2026 Edition

About This Part

Volume 6E converts AWS knowledge into an enterprise operating model. The focus is not merely on creating resources, but on creating them repeatedly, reviewing every change, deploying applications safely, detecting failure quickly, preserving audit evidence, enforcing security controls, and governing many accounts without blocking delivery teams.

The chapters combine Infrastructure as Code, continuous delivery, observability, operational automation, cryptographic controls, threat detection, perimeter protection, multi-account governance, resilience engineering, and FinOps. The capstone joins these disciplines into a secure landing zone and software delivery platform.

Scope and Safety

- Use sandbox accounts and budgets. Some services in this part can create continuous charges, including NAT gateways, log ingestion, tracing, Config recording, security services, managed firewalls, and resilience assessments.
- Use federation and temporary credentials. Do not store access keys in build specifications, pipeline variables, repositories, instance files, container images, or application packages.
- Test policies, controls, organization changes, and firewall rules in a staging OU or isolated account before broad enforcement. Governance mistakes can block recovery and administrative access.
- Encrypt logs and artifacts, restrict cross-account roles, validate artifact provenance, and separate build, security, shared-services, and workload accounts.
- Validate current AWS Regions, quotas, pricing, runtime support, service integrations, and control availability in official AWS documentation before production implementation.

Table of Contents

Chapter 41	AWS CloudFormation and Infrastructure as Code
Chapter 42	AWS CDK and Platform Engineering
Chapter 43	AWS CodePipeline and Release Orchestration
Chapter 44	AWS CodeBuild, CodeDeploy, and Deployment Strategies
Chapter 45	Amazon CloudWatch Observability Engineering
Chapter 46	Distributed Tracing, X-Ray, and Application Signals
Chapter 47	CloudTrail, AWS Config, and Audit Engineering
Chapter 48	AWS Systems Manager and Fleet Operations
Chapter 49	KMS, Secrets, Parameters, and Certificate Management
Chapter 50	Threat Detection, Vulnerability Management, and Security Hub
Chapter 51	AWS WAF, Shield, Firewall Manager, and Network Firewall
Chapter 52	AWS Organizations, Control Tower, and Landing Zones
Chapter 53	DevSecOps and Software Supply-Chain Security
Chapter 54	Well-Architected, Resilience Engineering, and FinOps
Chapter 55	Enterprise Landing Zone and Secure Delivery Capstone

Navigation Heading styles are applied so Word and LibreOffice can build a dynamic table of contents and provide outline navigation.

VOLUME 6E | CHAPTER 41

Chapter 41 - AWS CloudFormation and Infrastructure as Code

Declarative infrastructure, controlled change, drift management, and multi-account deployment

Learning Objectives

- Explain templates, stacks, resources, parameters, mappings, conditions, outputs, and intrinsic functions.
- Use change sets, rollback controls, deletion policies, stack policies, drift detection, nested stacks, and StackSets.
- Design safe promotion, modularization, testing, and recovery practices for production Infrastructure as Code.

Why This Matters in Production

Manual infrastructure changes are difficult to reproduce and audit. CloudFormation turns desired infrastructure state into versioned templates and gives teams a transaction-oriented deployment workflow. The engineering challenge is controlling replacement risk, preventing configuration drift, and keeping templates understandable as systems grow.

Production principle Treat every infrastructure change as a reviewed software release. Preview the change, protect stateful resources, validate policy, deploy progressively, and preserve rollback evidence.

CloudFormation Mental Model

CloudFormation receives a declarative template, resolves dependencies, calls AWS service APIs, and records the resulting resources as a stack. The template describes the desired state; CloudFormation determines much of the execution order from references and dependencies.

Element	Purpose	Production guidance
Resources	AWS objects to create or update	Use explicit names sparingly; allow generated names where replacement and parallel environments matter.
Parameters	Deployment-time inputs	Constrain types and allowed values; do not use parameters as an ungoverned configuration database.
Outputs	Values exposed after deployment	Export only stable interfaces; excessive cross-stack exports create coupling.
Conditions	Conditional resources or properties	Use for genuine environment variation, not to maintain unrelated architectures in one template.
Mappings	Static lookup data	Keep small and versioned; prefer Systems Manager public parameters for changing AMI identifiers.

Safe Change and Lifecycle Controls

Change sets show planned additions, modifications, removals, and replacements before execution. Stateful resources require extra controls because a replacement can cause downtime or data loss. Use `DeletionPolicy` and `UpdateReplacePolicy` intentionally, create backups, and understand which property changes require replacement.

- Apply stack policies to protect critical resources from unintended updates.
- Use termination protection for foundational stacks.
- Use rollback alarms for deployments that should automatically revert when health metrics deteriorate.
- Detect drift regularly and decide whether to revert the resource, import the actual state, or update the template.
- Use StackSets for governed deployment across accounts and Regions; stage operations and cap concurrency.

```
aws cloudformation validate-template --template-body file://network.yaml
aws cloudformation create-change-set \
  --stack-name prod-network \
  --change-set-name release-2026-07-31 \
  --change-set-type UPDATE \
  --template-body file://network.yaml \
  --capabilities CAPABILITY_NAMED_IAM
aws cloudformation describe-change-set --stack-name prod-network --change-set-name
release-2026-07-31
```

Template Architecture and Testing

Organize templates by lifecycle and ownership boundary: network, identity, data, application, and observability stacks often evolve at different rates. Use nested stacks or modules carefully, publish reusable patterns, lint templates, evaluate policies before deployment, and create ephemeral integration environments.

Anti-pattern A single giant stack containing every resource increases blast radius, makes updates slow, and couples unrelated teams. Split by ownership and lifecycle, not by arbitrary service count.

Production Design Checklist

- Templates are version-controlled, peer-reviewed, linted, and policy-checked.
- Change sets are reviewed before production execution.
- Stateful resources have backup, retention, and replacement controls.
- Drift detection is scheduled and exceptions are documented.
- StackSets use staged organizational-unit rollout and failure tolerance.
- Rollback, break-glass access, and recovery runbooks are tested.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
UPDATE_ROLLBACK_FAILED	Rollback action cannot complete	Inspect stack events from the first failure and identify blocked dependency	Fix the dependency, continue update rollback, or skip only the minimum resource with documented risk.
Unexpected replacement	Property change requires replacement	Review change set resource replacement field and service specification	Redesign with parallel resource and migration; protect data before execution.
Template exceeds limits	Large or deeply nested template	Measure template size and resource count	Split by lifecycle, use nested stacks or modules, and store packaged artifacts in S3.
Stack drift	Out-of-band manual changes	Run drift detection and compare expected versus actual properties	Revert manual change or update/import state through an approved process.

Hands-on Lab 41: Deploy and Review a Change Set

Objective Create a small network stack, update it through a reviewed change set, introduce controlled drift, and restore declarative ownership.

Implementation

1. Create a YAML template with a VPC, tags, parameter validation, and outputs.
2. Validate and deploy the initial stack with termination protection.
3. Change a non-destructive property and create a change set without executing it.
4. Review the resource-level plan and execute the approved change set.
5. Modify a tag manually, run drift detection, and record the drift result.
6. Restore the intended value through the template and verify IN_SYNC status.

Validation Evidence

- Stack events show successful create and update operations.
- The change set review identifies the expected modification only.
- Drift detection reports the manual change and later returns IN_SYNC.

Cleanup

- Delete the training stack after disabling termination protection.
- Remove packaged template artifacts from the training bucket.

Stretch challenge Add a stack policy and a rollback alarm, then demonstrate that a prohibited update or health regression is contained.

Chapter Summary

- CloudFormation manages AWS resources as stacks from declarative templates.
- Change sets, lifecycle policies, drift detection, and StackSets are core production controls.
- Template boundaries should follow ownership, lifecycle, and blast radius.

Interview Questions and Model Answers

1. Why use a change set?

It previews the actions CloudFormation intends to take, including replacements and deletions, before execution.

2. What is drift?

Drift is a difference between the actual resource configuration and the configuration represented by the stack template and parameters.

3. When should StackSets be used?

For centrally governed, repeatable stack deployment across multiple accounts and Regions.

Official AWS References

CloudFormation stacks: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacks.html>

Change sets: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-updating-stacks-changesets.html>

Drift detection: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-stack-drift.html>

StackSets: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html>

VOLUME 6E | CHAPTER 42

Chapter 42 - AWS CDK and Platform Engineering

Reusable constructs, environment bootstrapping, policy guardrails, and developer experience

Learning Objectives

- Explain CDK apps, stacks, constructs, synthesis, assets, contexts, and bootstrapping.
- Differentiate L1, L2, and L3 constructs and choose abstraction levels deliberately.
- Build governed platform constructs with tests, Aspects, policy checks, and release management.

Why This Matters in Production

The AWS CDK lets teams express infrastructure in familiar programming languages while provisioning through CloudFormation. This increases reuse and testability, but it can also hide infrastructure behavior behind abstractions. Platform teams must create a paved road that improves delivery without concealing security, cost, and replacement implications.

Production principle Use code to create safe abstractions, not to bypass infrastructure review. Synthesized CloudFormation remains the deployable contract and must be inspected and governed.

CDK Execution Model

A CDK application instantiates constructs inside stacks. The toolkit synthesizes CloudFormation templates and publishes required assets. Deployment uses a bootstrapped environment that typically contains roles, an artifact bucket, and supporting resources.

Construct level	Description	Use
L1	Direct representation of CloudFormation resource types	Use when higher-level support is unavailable or precise property access is required.
L2	Intent-oriented resource abstraction with sensible defaults	Default choice for most platform code; review generated resources and permissions.
L3	Pattern containing multiple coordinated resources	Use for approved common architectures with explicit extension points and observability.

Bootstrapping and Trust

Bootstrapping creates resources used by deployments. Treat the bootstrap stack as privileged infrastructure. Limit which accounts can publish assets or assume deployment roles, use modern bootstrap templates, version bootstrap environments, and separate developer synthesis from production deployment authorization.

```
npm install -g aws-cdk
cdk bootstrap aws://111122223333/ap-south-1
cdk synth
cdk diff
```

```
cdk deploy --require-approval broadening
```

Platform Construct Design

A good platform construct encodes required tags, encryption, logging, alarms, least-privilege roles, standard networking, and cost metadata. It also exposes controlled configuration rather than hundreds of raw service properties. Version constructs semantically, publish migration notes, and test synthesized templates.

- Use unit assertions to verify resources and security properties.
- Use snapshot tests carefully; prefer targeted assertions for meaningful behavior.
- Apply Aspects for cross-cutting checks and mutations, but avoid invisible surprise behavior.
- Run cdk-nag or equivalent policy checks and CloudFormation Guard in CI.
- Create integration tests that deploy into disposable accounts and verify runtime behavior.

Context and Environment Independence

Avoid reading live environment data during synthesis unless the lookup is intentionally cached and reviewed. Explicit account and Region context improves determinism. Separate configuration data from application code and avoid branching that creates radically different architectures from one stack.

Production Design Checklist

- Bootstrap roles and artifact buckets are restricted and monitored.
- Synthesized templates are archived as pipeline artifacts and reviewed.
- Constructs encode mandatory security, logging, tagging, and ownership defaults.
- Construct versions and breaking changes are managed deliberately.
- Tests verify resources, policies, and critical runtime behavior.
- Production deployment occurs through pipeline roles, not developer laptops.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Bootstrap version error	Environment bootstrap is missing or outdated	Inspect CDK bootstrap stack and toolkit message	Upgrade bootstrap stack through controlled change and verify trusted accounts.
Unexpected template diff	Context lookup, feature flag, or library version changed	Compare cdk.context.json, package lock, CDK version, and synthesized output	Pin dependencies and context; review and commit intended updates.
Asset publication denied	Publishing role or bucket/KMS policy blocks access	Inspect assumed role and CloudTrail denial	Correct bootstrap trust and least-privilege asset permissions.
Construct grants broad IAM	Convenience grant or pattern generated wildcard access	Inspect synthesized IAM policies and policy findings	Replace broad grant with scoped resource/action permissions or custom construct logic.

Hands-on Lab 42: Create a Governed CDK Construct

Objective Build a reusable encrypted S3 construct with mandatory tags, access logging, lifecycle rules, tests, and a deployment diff.

Implementation

1. Initialize a CDK v2 application in TypeScript or Python.
2. Create a construct that provisions an encrypted, versioned bucket with public access blocked and lifecycle rules.
3. Add required owner, environment, data-classification, and expiry tags.
4. Write assertions for encryption, versioning, access blocking, and lifecycle configuration.
5. Run synth and diff; inspect the generated CloudFormation and IAM actions.
6. Deploy to a sandbox account through a dedicated role and validate the bucket controls.

Validation Evidence

- All construct tests pass.
- Synthesized template contains expected controls and no public access.
- CloudTrail shows deployment through the intended role.

Cleanup

- Destroy the sandbox stack after emptying the bucket.
- Remove unused bootstrap assets only if approved; do not delete shared bootstrap infrastructure.

Stretch challenge Package the construct as a versioned internal library and add an Aspect that rejects unencrypted storage resources.

Chapter Summary

- CDK synthesizes CloudFormation and therefore inherits CloudFormation lifecycle behavior.
- Constructs are the unit of platform reuse and should encode secure defaults and controlled extension points.
- Bootstrapping, dependency pinning, template review, and policy tests are production requirements.

Interview Questions and Model Answers

1. What does `cdk synth` produce?

A CloudFormation template plus asset metadata required to deploy the defined stacks.

2. What is the difference between L1 and L2 constructs?

L1 closely maps to CloudFormation resources; L2 provides higher-level intent, defaults, helper methods, and relationships.

3. Why must the bootstrap stack be secured?

Its roles and artifact resources participate in deployments and can have broad permissions across target environments.

Official AWS References

CDK documentation: <https://docs.aws.amazon.com/cdk/>

Constructs: <https://docs.aws.amazon.com/cdk/v2/guide/constructs.html>

Bootstrapping: <https://docs.aws.amazon.com/cdk/v2/guide/bootstrapping.html>

CDK best practices: <https://docs.aws.amazon.com/cdk/v2/guide/best-practices.html>

VOLUME 6E | CHAPTER 43

Chapter 43 - AWS CodePipeline and Release Orchestration

Deterministic promotion, approvals, artifact integrity, and cross-account delivery

Learning Objectives

- Design CodePipeline stages, actions, artifacts, triggers, variables, conditions, and approvals.
- Build cross-account promotion with isolated roles and encrypted artifact stores.
- Define release evidence, rollback, concurrency, and failure-handling practices.

Why This Matters in Production

A release pipeline is a control system for software change. It must produce reproducible artifacts, enforce tests and policy, separate environments, preserve evidence, and stop unsafe releases. CodePipeline coordinates these stages while integrating AWS and third-party actions.

Production principle Build once, promote the same immutable artifact, and make every production transition observable, authorized, and reversible.

Pipeline Architecture

A production pipeline commonly separates source, build, security validation, integration test, staging deployment, operational verification, approval, production deployment, and post-deployment validation. Stages express ordering; actions within a stage may run in parallel when dependencies permit.

Stage	Primary evidence	Failure decision
Source	Commit identity, signed tag, pull-request review	Reject untrusted or unreviewed source.
Build	Test results, SBOM, artifact digest	Stop on test or provenance failure.
Security	Policy, dependency, secret, and image scan findings	Apply severity and exception policy.
Staging	Deployment status, synthetic and integration tests	Rollback or quarantine artifact.
Production	Approval record, change window, health metrics	Automatic rollback or incident response.

Artifacts, Encryption, and Cross-Account Roles

Use a versioned and encrypted artifact store. Each action role should receive only the permissions required for its provider and resources. In cross-account pipelines, the tooling account must be allowed to assume narrowly scoped deployment roles in workload accounts, and KMS key policies must permit the required artifact encryption workflow.

```
aws codepipeline get-pipeline --name commerce-release
aws codepipeline list-pipeline-executions --pipeline-name commerce-release --max-results 10
aws codepipeline get-pipeline-state --name commerce-release
aws codepipeline stop-pipeline-execution \
```

```
--pipeline-name commerce-release \
--pipeline-execution-id EXAMPLE-ID \
--abandon
```

Release Controls

Configure triggers intentionally to avoid duplicate execution. Define superseded-execution behavior, stage conditions, retry policy, manual approvals, and notification channels. Record artifact digests and deployment identifiers so an incident commander can identify exactly what code and infrastructure changed.

Production Design Checklist

- Pipeline definition is stored as IaC and reviewed.
- Build output is immutable and promoted without rebuilding per environment.
- Artifact bucket and KMS key policies support only intended principals.
- Cross-account deployment roles are narrow and externally auditable.
- Production approval, health verification, and rollback paths are explicit.
- Pipeline logs, events, notifications, and evidence retention meet audit requirements.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Pipeline waits indefinitely	Approval or action callback not completed	Inspect stage state, token-based action, and approval notification	Complete or reject action; configure timeout and escalation.
Cross-account artifact denied	KMS or S3 policy missing target role access	Inspect CloudTrail and artifact store policies	Add least-privilege key and bucket grants for action role.
Wrong version deployed	Artifact rebuilt or variable points to mutable tag	Compare artifact digest and execution metadata	Build once and promote digest-addressed artifacts.
Duplicate pipeline executions	Multiple triggers or retry automation	Inspect trigger rules and event history	Consolidate triggers and define execution mode deliberately.

Hands-on Lab 43: Create a Multi-Stage Release Pipeline

Objective Build a pipeline that packages an application once, validates it, deploys to staging, requires approval, and promotes the same artifact to production.

Implementation

1. Create encrypted versioned artifact storage and pipeline service roles.
2. Configure a source action from an approved repository connection.
3. Add a CodeBuild action that runs tests and writes an artifact manifest with commit ID and digest.
4. Deploy to a staging target and run synthetic validation.
5. Add a manual approval with change record and evidence link.
6. Deploy the identical artifact to a production sandbox and run post-deployment health checks.

Validation Evidence

- Artifact digest remains identical across staging and production.
- Approval and execution history are visible in pipeline evidence.
- Failure of the health check prevents or rolls back promotion.

Cleanup

- Delete sandbox deployment targets and pipeline after evidence capture.
- Retain or remove audit artifacts according to the training retention policy.

Stretch challenge Move production deployment into a separate AWS account and configure scoped artifact, KMS, and assume-role permissions.

Chapter Summary

- CodePipeline orchestrates ordered release actions and evidence.
- Cross-account delivery requires careful role, artifact, and KMS design.
- Immutable promotion and explicit rollback are central release-engineering controls.

Interview Questions and Model Answers

1. Why should an artifact not be rebuilt for production?

Rebuilding can change dependencies or outputs, so staging no longer validates the exact artifact promoted to production.

2. What is a cross-account action role?

A role in the target account assumed by the pipeline or action to perform a narrowly defined deployment.

3. What evidence should a pipeline preserve?

Source identity, artifact digest, tests, security findings, approvals, deployment IDs, health results, and rollback actions.

Official AWS References

CodePipeline capabilities: <https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome-what-can-I-do.html>

Getting started: <https://docs.aws.amazon.com/codepipeline/latest/userguide/getting-started-codepipeline.html>

CodePipeline best practices: <https://docs.aws.amazon.com/codepipeline/latest/userguide/best-practices.html>

VOLUME 6E | CHAPTER 44

Chapter 44 - AWS CodeBuild, CodeDeploy, and Deployment Strategies

Secure builds, reproducible artifacts, and controlled workload rollout

Learning Objectives

- Configure CodeBuild projects, buildspec phases, caches, reports, VPC access, and ephemeral credentials.
- Explain CodeDeploy applications, deployment groups, AppSpec files, hooks, and rollback.
- Choose in-place, rolling, blue/green, canary, and linear rollout patterns based on risk.

Why This Matters in Production

Build and deployment systems execute untrusted inputs with powerful credentials. They must be isolated, reproducible, observable, and constrained. CodeBuild creates artifacts in ephemeral environments; CodeDeploy coordinates application rollout to supported compute platforms.

Production principle Minimize build privilege, make artifacts reproducible, test lifecycle hooks, and shift traffic gradually when failure impact is high.

Secure Build Engineering

Build projects should use dedicated service roles, pinned runtime images, explicit dependency locks, encrypted artifacts, and isolated network access. Avoid privileged mode unless container image builds require it, and then restrict the project and role carefully. Store secrets in Secrets Manager or Parameter Store and mask output.

```
version: 0.2
phases:
  install:
    commands:
      - npm ci
  pre_build:
    commands:
      - npm test
      - npm audit --audit-level=high
  build:
    commands:
      - npm run build
  post_build:
    commands:
      - sha256sum dist/app.zip > dist/manifest.sha256
artifacts:
  files:
    - dist/**/*
reports:
  unit-tests:
    files:
      - reports/junit.xml
```

CodeDeploy Components

Component	Purpose	Operational focus
Application	Logical deployment target family	Separate compute platforms and ownership boundaries.
Deployment group	Target selection and deployment configuration	Alarms, rollback, traffic shifting, service role, tags.
Revision	Deployable content and metadata	Immutable version and verified origin.
AppSpec	Maps files, hooks, and target behavior	Test hooks under success, retry, rollback, and partial failure.
Lifecycle hooks	Pre/post deployment validation and control	Idempotent scripts, bounded timeouts, structured logs.

Deployment Strategy Selection

In-place deployment updates existing instances and may reduce capacity during rollout. Blue/green creates or selects a replacement environment, validates it, and shifts traffic. Lambda and ECS deployments can use canary or linear traffic shifting. Choose based on state, rollback speed, capacity cost, compatibility, and blast radius.

- Use automatic rollback on deployment failure and selected CloudWatch alarms.
- Keep database schema changes backward-compatible across old and new versions.
- Use bake time to detect delayed failures before full traffic shift.
- Test termination and cleanup behavior for the old environment.
- Record deployment group, revision, configuration, and target health as evidence.

Production Design Checklist

- Build role permissions are resource-scoped and temporary.
- Dependencies and toolchains are pinned and verified.
- Secrets are retrieved at runtime and not printed or stored in artifacts.
- Deployment hooks are idempotent and time-bounded.
- Alarms and automatic rollback are configured and tested.
- Schema and API changes support mixed-version operation during rollout.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Build cannot reach dependency	VPC route, proxy, DNS, or endpoint issue	Inspect build network configuration and logs	Add controlled egress or VPC endpoints; avoid broad internet access.
Build cache causes stale output	Cache key is insufficient or dependencies changed	Compare lock file and cache paths	Version cache keys or disable cache for critical reproducibility.
Deployment hook timeout	Script hangs or dependency unavailable	Inspect lifecycle event logs and agent output	Add bounded timeout, retries, and health prerequisite checks.
Blue/green rollback fails	Old environment removed too early or schema incompatible	Review traffic and termination settings	Increase bake/retention and use backward-compatible migration pattern.

Hands-on Lab 44: Build and Blue/Green Deploy an Application

Objective Use CodeBuild to create a verified artifact and CodeDeploy to perform a monitored blue/green deployment with rollback.

Implementation

1. Create a least-privilege CodeBuild role and build project.
2. Add unit tests, dependency checks, artifact hashing, and test reports to buildspec.yml.
3. Configure a deployment application and group with health alarms and automatic rollback.
4. Create an AppSpec file and idempotent validation hooks.
5. Deploy a healthy revision and capture traffic-shift evidence.
6. Deploy an intentionally unhealthy revision and verify alarm-driven rollback.

Validation Evidence

- Build report and artifact hash are retained.
- Healthy revision completes and target health remains stable.
- Unhealthy revision rolls back automatically with clear event history.

Cleanup

- Delete sandbox deployment group, application, build project, and temporary targets.
- Remove artifacts according to retention policy.

Stretch challenge Add a canary or linear deployment configuration and compare error-budget consumption with an all-at-once release.

Chapter Summary

- CodeBuild provides ephemeral build execution and report integration.
- CodeDeploy supports controlled rollout and automatic rollback across supported compute platforms.
- Build privilege, artifact reproducibility, health alarms, and compatible changes determine deployment safety.

Interview Questions and Model Answers

1. When is privileged mode used in CodeBuild?

Commonly for building container images that require Docker-in-Docker behavior; it increases risk and should be narrowly controlled.

2. What does an AppSpec file define?

Deployment files, destinations, permissions, and lifecycle hooks or traffic-related functions depending on compute platform.

3. Why are backward-compatible database changes important?

During gradual rollout, old and new application versions may run simultaneously against the same database.

Official AWS References

CodeBuild with CodePipeline: <https://docs.aws.amazon.com/codebuild/latest/userguide/how-to-create-pipeline.html>

CodeDeploy deployments: <https://docs.aws.amazon.com/codedeploy/latest/userguide/deployments.html>

CodeDeploy getting started: <https://docs.aws.amazon.com/codedeploy/latest/userguide/getting-started-codedeploy.html>

VOLUME 6E | CHAPTER 45

Chapter 45 - Amazon CloudWatch Observability Engineering

Metrics, logs, alarms, dashboards, synthetics, and actionable operations

Learning Objectives

- Design metrics, dimensions, logs, retention, queries, dashboards, and alarms.
- Explain metric math, anomaly detection, composite alarms, Container Insights, and Database Insights.
- Create observability that supports service-level objectives and incident response rather than dashboard decoration.

Why This Matters in Production

Observability gives operators evidence about system state from outputs such as metrics, logs, traces, synthetic tests, and events. CloudWatch is a broad telemetry and action platform. Poor designs either miss failures or generate alert fatigue and excessive cost.

Production principle Alert on user impact and actionable risk. Retain high-cardinality detail where it is useful, aggregate where it is not, and connect every alarm to ownership and a runbook.

Telemetry Model

Metrics are numeric time series identified by namespace, name, and dimensions. Logs contain event records and can be searched or transformed into metrics. Alarms evaluate metric expressions and can trigger notifications or automated actions. Dashboards provide shared operational context.

Signal	Best use	Common mistake
Metric	Fast aggregation, alerting, SLO indicators	Creating unbounded high-cardinality dimensions.
Log	Detailed event and diagnostic evidence	Logging secrets or retaining everything forever.
Synthetic canary	External user-path validation	Testing only infrastructure health, not business transactions.
Dashboard	Incident and service review context	Displaying many signals without thresholds, ownership, or decisions.

Alarm Engineering

Use multiple evaluation periods to balance speed and noise. Treat missing data based on the expected signal. Composite alarms suppress secondary symptoms and express service-level conditions. Metric math can derive rates and ratios such as errors per request. Anomaly detection helps when fixed thresholds are unsuitable, but operators still need business context.

```
aws cloudwatch put-metric-alarm \
  --alarm-name CheckoutHighErrorRate \
  --namespace Commerce/Checkout \
  --metric-name ErrorRate \
```

```
--statistic Average \
--period 60 \
--evaluation-periods 5 \
--datapoints-to-alarm 3 \
--threshold 2 \
--comparison-operator GreaterThanThreshold \
--treat-missing-data notBreaching
```

Logs and Cost Control

Define structured JSON logs with timestamp, severity, service, environment, request ID, trace ID, outcome, and safe business identifiers. Apply retention by data class. Use subscription filters only where a real-time downstream use case exists. Monitor ingestion volume, query behavior, custom metric count, and high-resolution metric usage.

Service-Level Objectives

A practical SLO starts with a user journey, a service-level indicator, a target, and an error budget. Example: 99.9 percent of checkout attempts complete successfully within an agreed latency over 30 days. Alarms should reflect rapid and sustained error-budget consumption rather than every isolated error.

Production Design Checklist

- Telemetry schema and ownership are documented.
- Logs are structured, sanitized, encrypted, and retained by policy.
- Alarms are actionable, tested, and linked to runbooks.
- Dashboards show traffic, errors, latency, saturation, changes, and dependencies.
- Synthetic checks validate critical user journeys from relevant locations.
- CloudWatch ingestion, storage, query, and custom metric costs are monitored.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Alarm never fires	Wrong dimensions, statistic, period, or missing-data behavior	Graph exact metric and inspect alarm history	Correct dimensions and evaluation settings; publish test data.
Alert storm	Many symptom alarms fire from one dependency	Correlate timeline and dependency graph	Use composite alarms, deduplication, and root-cause-oriented routing.
Logs missing	Agent, role, log group, network, or configuration issue	Check agent status, permissions, and endpoint connectivity	Fix role, endpoint, config, and restart collection safely.
Unexpected CloudWatch cost	High ingestion, retention, queries, custom metrics, or traces	Use cost allocation and usage metrics	Reduce noise, aggregate, sample, shorten retention, and remove unused telemetry.

Hands-on Lab 45: Build an Actionable Service Dashboard

Objective Instrument a sample service with structured logs and custom metrics, then create tested alarms, a dashboard, and a synthetic transaction.

Implementation

1. Define service ownership, user journey, SLI, threshold, and runbook link.
2. Publish structured application logs with request and trace identifiers.
3. Create request, error, latency, and saturation metrics.
4. Build a dashboard with deployment annotations and dependency health.
5. Create a composite alarm and notification route.
6. Inject latency and errors, verify alarm behavior, and record incident evidence.

Validation Evidence

- Dashboard reflects the injected failure within expected delay.
- Alarm changes state only after configured evaluation criteria.
- Runbook and owner information are accessible from the alert.

Cleanup

- Delete training alarms, dashboard, canary, and custom metrics source.
- Apply short retention or delete training log groups.

Stretch challenge Define a burn-rate alarm for a 30-day availability SLO and compare it with a fixed error-rate threshold.

Chapter Summary

- CloudWatch combines metrics, logs, alarms, dashboards, and application monitoring.
- Observability should answer operational questions and support decisions.
- Alert quality, structured telemetry, retention, and cost controls are engineering responsibilities.

Interview Questions and Model Answers

1. What is the difference between monitoring and observability?

Monitoring checks known conditions; observability uses system outputs to investigate known and previously unknown internal states.

2. Why use a composite alarm?

It combines or suppresses underlying alarm conditions to reduce noise and express a higher-level service condition.

3. What causes high-cardinality metric problems?

Dimensions with many unique values create many time series and increase cost and operational complexity.

Official AWS References

CloudWatch documentation: <https://docs.aws.amazon.com/cloudwatch/>

Application monitoring: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Application-Monitoring-Intro.html>

Application Signals: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Application-Monitoring-Sections.html>

VOLUME 6E | CHAPTER 46

Chapter 46 - Distributed Tracing, X-Ray, and Application Signals

End-to-end request visibility, service maps, SLOs, and trace-informed debugging

Learning Objectives

- Explain traces, spans, context propagation, sampling, service maps, annotations, and metadata.
- Use AWS X-Ray, OpenTelemetry, and CloudWatch Application Signals for application performance monitoring.
- Correlate traces with logs, metrics, synthetic tests, and deployments.

Why This Matters in Production

Modern requests cross gateways, functions, containers, queues, databases, and third-party APIs. Component metrics may look healthy while an end-to-end transaction fails. Distributed tracing follows the request path and reveals latency, errors, retries, and dependency behavior.

Production principle Propagate context across every boundary, sample intentionally, protect sensitive data, and use traces to explain service-level symptoms rather than collect them without purpose.

Trace Model

A trace represents an end-to-end transaction. Spans represent units of work such as an HTTP call, database query, function invocation, or message processing step. Trace context must be propagated across synchronous and asynchronous boundaries. Annotations support indexed filtering; metadata contains detail that is not indexed.

Design choice	Guidance	Risk
Sampling	Keep representative baseline plus elevated capture for errors or investigations	Too little hides rare failure; too much increases cost and data exposure.
Annotations	Use bounded, non-sensitive values for search	High-cardinality or personal data causes cost and privacy problems.
Span boundaries	Align to meaningful operations and dependencies	Too many tiny spans create noise; too few hide latency.
Context propagation	Carry trace identifiers through HTTP and messaging	Broken propagation fragments the service graph.

Instrumentation Options

Applications can send trace data using AWS Distro for OpenTelemetry, OpenTelemetry SDKs and collectors, X-Ray-compatible instrumentation, or CloudWatch Application Signals setup. Standardize libraries and collector configuration through platform tooling. Use semantic conventions and resource attributes consistently.

Application Signals and SLO Context

Application Signals can surface services, operations, dependencies, latency, faults, errors, and service-level objectives. It complements logs and metrics; it does not remove the need for domain-specific telemetry. Define service boundaries carefully and ensure workload identity and environment attributes are correct.

```
aws xray get-service-graph --start-time 2026-07-31T00:00:00Z --end-time 2026-07-31T01:00:00Z
aws xray get-trace-summaries --start-time 2026-07-31T00:00:00Z --end-time 2026-07-31T01:00:00Z --filter-expression "service('checkout') AND error = true"
```

Production Design Checklist

- Trace context crosses HTTP, messaging, and background-job boundaries.
- Sampling rules reflect traffic, risk, and cost.
- Sensitive data is excluded from span names, attributes, annotations, and metadata.
- Logs include trace and span identifiers for correlation.
- Service names and environment attributes are consistent.
- Tracing failure does not block the business transaction.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Broken trace	Context not propagated across library or message boundary	Compare trace headers and span parent IDs	Instrument boundary and carry context in supported message attributes.
Service map duplicates service	Inconsistent service names or resource attributes	Inspect instrumentation configuration	Standardize naming and deployment attributes.
No traces	Sampling, permissions, collector, network, or instrumentation failure	Check application and collector logs plus IAM	Fix export endpoint, role, sampling rule, and collector configuration.
Tracing increases latency	Synchronous export, excessive spans, or resource pressure	Profile exporter and collector	Use batch export, appropriate sampling, and bounded attributes.

Hands-on Lab 46: Trace a Distributed Transaction

Objective Instrument an API and worker path, propagate trace context through a queue, and correlate a slow dependency with logs and metrics.

Implementation

1. Deploy or use a sandbox API that sends work to a queue and worker.
2. Enable OpenTelemetry or Application Signals instrumentation.
3. Include trace identifiers in structured logs.
4. Create a controlled slow downstream dependency.
5. Search traces for the affected operation and inspect the service map.
6. Correlate the trace with latency metrics, logs, and deployment time.

Validation Evidence

- One trace shows API, queue, worker, and dependency spans.
- Slow dependency is visible as the dominant duration.
- Trace ID retrieves corresponding logs.

Cleanup

- Remove training instrumentation configuration and sandbox resources.
- Confirm short retention or deletion for test telemetry.

Stretch challenge Implement dynamic or tail-based sampling in a collector so errors and high-latency requests receive higher retention.

Chapter Summary

- Distributed tracing explains request behavior across service boundaries.
- Context propagation, naming, sampling, and privacy determine trace usefulness.
- Application Signals connects service operations, dependencies, and SLO context with CloudWatch.

Interview Questions and Model Answers

1. What is a span?

A timed unit of work within a trace, with parent-child relationships and attributes describing the operation.

2. What is the purpose of sampling?

To control telemetry volume and cost while retaining representative and important traces.

3. Why include trace IDs in logs?

It allows an operator to pivot from an aggregate symptom to one transaction and its detailed event evidence.

Official AWS References

CloudWatch Application Signals setup: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Getting-Started-App-Signals.html>

X-Ray getting started: <https://docs.aws.amazon.com/xray/latest/devguide/xray-gettingstarted.html>

CloudWatch and X-Ray integration: <https://docs.aws.amazon.com/xray/latest/devguide/xray-services-cloudwatch.html>

VOLUME 6E | CHAPTER 47

Chapter 47 - CloudTrail, AWS Config, and Audit Engineering

API history, configuration evidence, compliance evaluation, and investigation readiness

Learning Objectives

- Differentiate CloudTrail event history, trails, Lake event data stores, management events, data events, and Insights events.
- Use AWS Config recorders, configuration history, rules, conformance packs, and aggregators.
- Design immutable audit evidence, delegated administration, retention, and investigation workflows.

Why This Matters in Production

Security and operations teams must answer who changed what, when, from where, and what configuration existed at the time. CloudTrail records AWS API activity; AWS Config records supported resource configuration and evaluates it against rules. Together they create critical evidence for investigations and compliance.

Production principle Centralize audit evidence in a dedicated account, protect it from workload administrators, validate integrity, and test retrieval before an incident.

CloudTrail Evidence

Event history provides recent management-event visibility within a Region, but production audit requires deliberately configured trails or CloudTrail Lake. Organization trails centralize coverage. Data events such as object or function activity can be high-volume and should use advanced selectors based on risk.

- Enable all required Regions and include global service events where appropriate.
- Encrypt log destinations and restrict deletion and policy changes.
- Use log file validation when trails deliver to S3.
- Monitor trail status and unauthorized changes.
- Choose retention from legal, investigation, and cost requirements.

AWS Config Model

AWS Config records configuration items and relationships for supported resources. Rules evaluate compliance; conformance packs bundle rules and remediation patterns; aggregators provide multi-account and multi-Region views. Config is not a replacement for CloudTrail because it focuses on resource state rather than every API event.

Audit Architecture

Need	Primary service	Evidence
Who called an API	CloudTrail	Identity, source, request parameters, response, time.
What was resource configuration	AWS Config	Configuration item and relationships at a point in time.
Continuous control status	Config rules / Security Hub CSPM	Compliant or noncompliant finding and evaluation.
Control evidence collection	Audit Manager and service evidence	Mapped evidence for assessment

Need	Primary service	Evidence
		frameworks.

```
aws cloudtrail get-trail-status --name organization-trail
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=StopLogging
aws configservice describe-configuration-recorder-status
aws configservice get-compliance-summary-by-config-rule
```

Production Design Checklist

- Organization-level CloudTrail and Config coverage is documented and monitored.
- Logs and configuration snapshots are encrypted and delivered to dedicated accounts.
- Data event selectors target high-risk resources without uncontrolled volume.
- Delegated administrator roles and access paths are separated from workloads.
- Retention, legal hold, integrity, and deletion controls are tested.
- Investigation queries and evidence export procedures are rehearsed.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Trail appears healthy but events missing	Region, event selector, resource type, or trail scope mismatch	Inspect trail configuration and event source	Correct Region and selectors; validate with a known test API call.
Config shows no resources	Recorder or delivery channel stopped, role denied, unsupported type	Check recorder status and service-linked role	Start recorder, correct role/channel, and verify supported resource coverage.
Audit bucket access denied	Bucket policy, KMS policy, organization condition, or role issue	Inspect S3/KMS denial in CloudTrail	Correct least-privilege delivery and read roles.
Excessive audit cost	Broad data events, Config scope, long retention, or frequent rules	Analyze usage by selector and resource type	Narrow to risk-based scope while preserving compliance evidence.

Hands-on Lab 47: Create a Central Audit Trail and Config Rule

Objective Configure a sandbox trail and Config recorder, generate a change, and correlate API identity with configuration state.

Implementation

1. Create encrypted audit storage with restricted write and read paths.
2. Create a trail for management events and one targeted data-event selector.
3. Enable Config recording for selected resource types.
4. Apply a managed or custom rule to evaluate an encryption control.
5. Perform a known resource change with a named role.

6. Retrieve the CloudTrail event and Config timeline for the same resource.

Validation Evidence

- CloudTrail identifies the role, action, time, and request.
- Config timeline shows before-and-after resource state.
- Rule evaluation records the expected compliance transition.

Cleanup

- Delete sandbox rules and trail if not shared; empty and delete temporary buckets.
- Stop and remove the training recorder only in the isolated account.

Stretch challenge Create an EventBridge rule that detects attempts to stop logging and routes a high-priority alert to the security team.

Chapter Summary

- CloudTrail records API activity; AWS Config records resource configuration and compliance state.
- Organization-level centralization and tamper resistance are core audit controls.
- Audit data volume and retention must be risk-based and tested.

Interview Questions and Model Answers

1. What is the difference between CloudTrail and Config?

CloudTrail records API activity; Config records supported resource state, relationships, and rule evaluations.

2. Why use an organization trail?

It centrally applies trail coverage to accounts in an AWS Organization and simplifies consistent audit collection.

3. What are data events?

High-volume resource-level operations such as S3 object access or Lambda invocation that are configured separately from management events.

Official AWS References

CloudTrail settings: <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-settings.html>

Organization trail: <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/creating-trail-organization.html>

AWS Config documentation: <https://docs.aws.amazon.com/config/>

Cloud governance decision guide: <https://docs.aws.amazon.com/pdfs/decision-guides/latest/cloud-governance-on-aws-how-to-choose/cloud-governance-on-aws-how-to-choose.pdf>

VOLUME 6E | CHAPTER 48

Chapter 48 - AWS Systems Manager and Fleet Operations

Secure node access, patching, automation, inventory, and operational control at scale

Learning Objectives

- Explain managed nodes, SSM Agent, Session Manager, Run Command, Patch Manager, Inventory, State Manager, Automation, and Parameter Store.
- Replace routine SSH/RDP administration with audited, policy-controlled access.
- Design fleet operations with maintenance windows, concurrency, failure thresholds, and rollback.

Why This Matters in Production

Operating many servers manually produces inconsistent state, unmanaged credentials, and weak evidence. Systems Manager provides a control plane for inventory, access, configuration, patching, and automation across supported AWS and hybrid nodes.

Production principle Prefer declarative and automated fleet operations. Human access should be temporary, recorded where required, and limited to exceptional diagnosis or repair.

Managed Node Foundation

A managed node requires a supported agent, network connectivity to Systems Manager endpoints, and an IAM role or activation that grants required permissions. Private subnets can use interface VPC endpoints to avoid internet egress. Inventory and compliance data depend on healthy agent communication.

Secure Access and Commands

Capability	Purpose	Control
Session Manager	Interactive shell or port forwarding without inbound SSH/RDP	IAM authorization, optional session logging, KMS, endpoint restrictions.
Run Command	Execute non-interactive commands at scale	Document allowlist, target tags, concurrency, error threshold.
Automation	Multi-step operational workflow	Approvals, role separation, rollback steps, idempotency.
State Manager	Maintain desired configuration association	Rate control, schedules, compliance reporting.
Patch Manager	Baseline-driven scanning and installation	Patch groups, maintenance windows, reboot policy, exception process.

Patch and Change Operations

Classify fleets by environment, criticality, application, maintenance window, and patch baseline. Patch nonproduction first, measure health, then progress to production cohorts. Use maximum concurrency and maximum errors to contain blast radius. Coordinate application draining and reboot with workload owners.

```
aws ssm describe-instance-information
aws ssm start-session --target i-0123456789abcdef0
aws ssm send-command \
  --document-name AWS-RunShellScript \
  --targets Key=tag:PatchGroup,Values=canary \
  --parameters commands="uname -a" \
  --max-concurrency 10% --max-errors 1
```

Production Design Checklist

- Managed-node roles are minimal and separate from application roles.
- Session logging and encryption meet policy requirements.
- Run Command and Automation documents are approved and versioned.
- Patch baselines, groups, windows, canaries, and exceptions are documented.
- VPC endpoints and DNS support private fleet management.
- Operational automation has rate control, rollback, and owner notification.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Node not managed	Agent stopped, role missing, clock/network/DNS issue	Check agent log, role, endpoints, and time sync	Restore connectivity/role, update agent, and re-register if required.
Session Manager unavailable	Plugin, IAM, endpoint, or agent capability issue	Inspect client and agent errors	Correct user permission, endpoint policy, and agent version.
Run Command delayed	Node offline, queueing, rate control, or plugin failure	Check command invocation per target	Restore node health or adjust safe concurrency.
Patch compliance inconsistent	Wrong patch group, baseline, snapshot, window, or reboot state	Review association and patch logs	Correct tags/baseline and rerun scan or controlled install.

Hands-on Lab 48: Operate a Private EC2 Fleet Without SSH

Objective Manage a private instance using Session Manager, Run Command, Inventory, and a controlled patch scan.

Implementation

1. Launch a private sandbox instance with no inbound administrative rule.
2. Attach a minimal Systems Manager managed-node role and required endpoints.
3. Verify the node appears in managed node inventory.
4. Start a Session Manager session and record authorization evidence.
5. Run an inventory or diagnostic command through Run Command with rate control.
6. Perform a patch scan and inspect compliance without installing production-impacting updates.

Validation Evidence

- No public IP or inbound SSH/RDP rule exists.

- Session and command activity appears in audit evidence.
- Patch compliance and inventory data are visible.

Cleanup

- Terminate the training instance and delete unused endpoints if dedicated.
- Remove temporary logs according to retention policy.

Stretch challenge Create an Automation runbook that snapshots an instance, drains it from a target group, patches it, validates health, and restores traffic.

Chapter Summary

- Systems Manager centralizes secure access, fleet inventory, patching, and automation.
- Session Manager reduces reliance on inbound administrative ports and persistent keys.
- Rate control, canaries, maintenance windows, and rollback make fleet changes safer.

Interview Questions and Model Answers

1. What is required for an EC2 instance to become a managed node?

A functioning SSM Agent, network access to Systems Manager endpoints, and an IAM role with required permissions.

2. Why prefer Session Manager to SSH?

It can avoid public access and inbound ports while using IAM authorization and centralized audit controls.

3. What do max concurrency and max errors control?

They limit rollout rate and stop a command or automation when failures exceed the allowed threshold.

Official AWS References

Systems Manager documentation: <https://docs.aws.amazon.com/systems-manager/>

Parameter Store reference: <https://docs.aws.amazon.com/systems-manager/latest/userguide/what-is-a-parameter.html>

Parameter Store working guide: <https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-working-with.html>

VOLUME 6E | CHAPTER 49

Chapter 49 - KMS, Secrets, Parameters, and Certificate Management

Cryptographic ownership, secret lifecycle, runtime configuration, and trusted transport

Learning Objectives

- Explain KMS keys, key policies, grants, encryption context, envelope encryption, and rotation.
- Choose between Secrets Manager and Parameter Store for secret and configuration lifecycles.
- Use AWS Certificate Manager for managed TLS certificates and design safe renewal and deployment.

Why This Matters in Production

Encryption is only as strong as its key ownership and access model. Secrets and certificates have lifecycles that include creation, distribution, rotation, revocation, recovery, and audit. Central platforms must make secure use simple while preventing uncontrolled decryption and secret sprawl.

Production principle Applications receive secrets just in time through workload identity. Humans and pipelines should not transport plaintext credentials, private keys, or long-lived tokens.

KMS Authorization Model

KMS key access is controlled by key policies and may also involve IAM policies and grants. A key policy must preserve administrative recovery while separating key administrators from data users. Encryption context can bind ciphertext use to a workload or resource context and appears in CloudTrail, so it must not contain secrets.

Capability	Use	Caution
Customer managed KMS key	Control policy, rotation, aliases, and lifecycle	Deletion has broad irreversible impact after waiting period.
AWS managed key	Service-managed encryption key for a service	Less policy and lifecycle control.
Grant	Delegated key use, often for AWS services	Inventory and revoke when no longer required.
Multi-Region key	Related key material in multiple Regions	Use only for architectures requiring compatible cryptographic identity.

Secrets Manager vs Parameter Store

Requirement	Secrets Manager	Parameter Store
Automatic managed rotation	Purpose-built support	Custom automation required.
Secret version staging	Built-in labels and versions	Parameter versions without the same rotation workflow.
General hierarchical config	Possible but cost may be higher	Strong fit for configuration and

Requirement	Secrets Manager	Parameter Store
		SecureString values.
Cross-service retrieval	Broad integrations	Broad integrations, including references to Secrets Manager.
Use case	Database credentials, API keys, rotated secrets	Configuration, AMI IDs, feature values, lightweight secrets.

Runtime Retrieval and Rotation

Use task roles, pod identities, Lambda execution roles, or instance roles to retrieve secrets. Cache only for a bounded period and handle rotation without restart where possible. Avoid environment variables for highly sensitive values when process inspection, crash dumps, or diagnostics may expose them.

Certificates and TLS

ACM can issue and renew eligible public certificates used by integrated AWS services. DNS validation is generally automation-friendly. For private PKI needs, evaluate ACM Private CA and its cost and governance. Monitor certificate expiry and failed renewal conditions, and separate certificate ownership by domain and environment.

Production Design Checklist

- KMS key policies have separate administration and usage roles.
- Deletion, disablement, and key-policy changes require strong approval and alerts.
- Secrets are rotated and consumers tolerate version transition.
- No secret is stored in source, image, build log, or unencrypted artifact.
- Configuration hierarchy, naming, ownership, and parameter policies are standardized.
- Certificates use automated validation and expiry monitoring.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Decrypt denied	Key policy, IAM, grant, encryption context, or Region mismatch	Inspect CloudTrail KMS event and key policy	Correct the specific authorization path; avoid broad wildcard decrypt.
Rotated secret breaks app	Consumer cached old value or rotation steps incomplete	Inspect secret versions and application retry	Support staged versions, refresh cache, and fix rotation contract.
Parameter not found	Path, Region, account, tier, or IAM mismatch	List path and caller identity	Correct fully qualified name, Region, and role permission.
ACM certificate pending	DNS validation record missing or wrong zone	Inspect validation record and DNS delegation	Create exact CNAME in authoritative zone and wait for propagation.

Hands-on Lab 49: Implement Runtime Secret Retrieval and Rotation

Objective Store a sandbox credential securely, retrieve it through a workload role, rotate it, and verify the application transitions without exposing plaintext.

Implementation

1. Create a customer managed KMS key with separated admin and usage roles.
2. Create a secret or SecureString parameter encrypted by the key.
3. Grant a sandbox workload role read/decrypt access only to that value.
4. Retrieve the value at runtime and confirm logs redact it.
5. Create a second version or perform rotation and test consumer refresh.
6. Review CloudTrail for GetSecretValue or GetParameter and KMS decrypt activity.

Validation Evidence

- Unauthorized role receives access denied.
- Authorized workload retrieves the current value without plaintext in logs.
- Rotation completes and the consumer uses the new value.

Cleanup

- Delete or schedule deletion for sandbox secret and key according to safe waiting-period rules.
- Remove the workload role and test application.

Stretch challenge Add cross-account secret consumption through a resource policy and KMS key policy while preventing direct human retrieval.

Chapter Summary

- KMS controls cryptographic use through key policies, IAM, and grants.
- Secrets Manager and Parameter Store serve overlapping but distinct lifecycle needs.
- Managed certificates, rotation, and workload identity reduce credential exposure.

Interview Questions and Model Answers

1. Why can an IAM allow still fail for KMS?

The KMS key policy and other authorization conditions must also permit the operation.

2. When is Secrets Manager preferable to Parameter Store?

When purpose-built secret lifecycle features such as managed rotation and version staging are needed.

3. What is envelope encryption?

Data is encrypted with a data key, and the data key is encrypted under a KMS key, improving performance and key management.

Official AWS References

Parameter Store: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

Secrets Manager integration:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_parameterstore.html

Cryptography decision guide: <https://docs.aws.amazon.com/pdfs/decision-guides/latest/cryptography-on-aws-how-to-choose/cryptography-on-aws-how-to-choose.pdf>

AWS KMS documentation: <https://docs.aws.amazon.com/kms/>

VOLUME 6E | CHAPTER 50

Chapter 50 - Threat Detection, Vulnerability Management, and Security Hub

GuardDuty, Inspector, Macie, centralized findings, and response prioritization

Learning Objectives

- Explain the roles of GuardDuty, Inspector, Macie, Security Hub, and Detective.
- Design organization-wide delegated administration and finding aggregation.
- Build triage, enrichment, suppression, remediation, and evidence workflows.

Why This Matters in Production

Security teams need continuous visibility across identities, workloads, software vulnerabilities, sensitive data, and cloud configuration. Individual services produce specialized findings; Security Hub can aggregate, normalize, correlate, and prioritize signals for centralized operations.

Production principle A finding is not a response. Establish ownership, context, severity policy, service-level targets, automated containment boundaries, and closure evidence.

Service Responsibilities

Service	Primary focus	Typical signal
Amazon GuardDuty	Threat detection from AWS data sources and runtime signals	Credential misuse, anomalous API activity, malware or workload threats.
Amazon Inspector	Automated vulnerability management	Package vulnerabilities, network exposure, code or image findings for supported resources.
Amazon Macie	Sensitive data discovery and S3 security	Potential sensitive data and risky bucket posture.
Security Hub CSPM / Security Hub	Security posture, controls, findings aggregation and correlation	Control findings and normalized findings from integrated services.
Amazon Detective	Investigation and relationship analysis	Entity behavior and contextual investigation graphs.

Organization Architecture

Designate dedicated delegated administrator accounts where supported. Enable services across accounts and Regions through Organizations integration or controlled automation. Centralize findings while preserving workload-account context. Keep the management account free of routine security operations.

Triage and Automation

Normalize findings into a workflow with status, owner, severity, asset criticality, exposure, exploitability, and business context. Use EventBridge to route findings. Automate low-risk enrichment and containment, but require human authorization for actions that could interrupt critical services unless emergency policy says otherwise.

- Suppress only with documented rationale, owner, and expiry.
- Track coverage gaps as findings, not only detected threats.
- Measure mean time to acknowledge, contain, remediate, and validate.
- Correlate software vulnerabilities with runtime exposure and asset importance.
- Preserve original finding, automation action, and closure evidence.

Production Design Checklist

- Security services are enabled across required accounts and Regions.
- Delegated administrators and aggregation Regions are documented.
- Findings route to a durable ticket or case system with ownership.
- Severity combines provider score with business and exposure context.
- Automated actions are idempotent, reversible, and tested.
- Suppression and exception records expire and receive review.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Member account missing findings	Organization enrollment, Region, delegated admin, or service not enabled	Check coverage and membership status	Enable service and aggregation consistently; remediate coverage finding.
Duplicate findings	Multiple integrations or ticket creation paths	Compare finding IDs and event rules	Deduplicate on stable finding identity and workflow state.
Inspector reports stale package	Image/runtime inventory lag or package metadata issue	Compare scan time and deployed artifact digest	Rescan current artifact and verify deployment inventory.
Macie cost unexpectedly high	Discovery scope or sampling broader than intended	Review job and automated discovery settings	Prioritize high-risk buckets and tune scope/retention.

Hands-on Lab 50: Build a Security Finding Triage Flow

Objective Enable sample findings, route them through EventBridge, enrich with tags, and create a controlled remediation record.

Implementation

1. Enable or use sandbox instances of GuardDuty, Inspector, Macie, and Security Hub where available.
2. Generate official sample findings rather than creating real malicious activity.
3. Create an EventBridge rule for high-severity findings.
4. Invoke a Lambda or Step Functions workflow that enriches the finding with owner and environment tags.
5. Create a ticket or structured record and send a notification.
6. Mark the sample finding workflow complete and retain evidence.

Validation Evidence

- Sample finding appears in the central service and event route.

- Enrichment attaches resource owner and environment.
- Workflow preserves finding ID, timestamps, actions, and closure state.

Cleanup

- Disable training event routes and delete temporary workflow resources.
- Keep security services enabled only according to account policy; do not disable shared organization controls.

Stretch challenge Add reversible quarantine for a tagged sandbox instance, with approval required for production-tagged resources.

Chapter Summary

- GuardDuty, Inspector, and Macie specialize in threats, vulnerabilities, and sensitive data.
- Security Hub centralizes posture and security findings across services and accounts.
- Coverage, prioritization, ownership, automation, and closure evidence determine operational value.

Interview Questions and Model Answers

1. How does GuardDuty differ from Inspector?

GuardDuty detects threats and anomalous activity; Inspector focuses on vulnerability and exposure assessment for supported workloads.

2. Why use a delegated administrator?

It moves service administration from the Organizations management account into a dedicated security account and centralizes operations.

3. What should influence finding priority beyond severity?

Asset criticality, public exposure, exploitability, active use, data classification, and compensating controls.

Official AWS References

Security Hub services: <https://docs.aws.amazon.com/securityhub/latest/userguide/what-are-securityhub-services.html>

Security Hub introduction: <https://docs.aws.amazon.com/securityhub/latest/userguide/what-is-securityhub-v2.html>

Security Hub control reference: <https://docs.aws.amazon.com/securityhub/latest/userguide/securityhub-controls-reference.html>

GuardDuty integration: <https://docs.aws.amazon.com/guardduty/latest/ug/securityhub-integration.html>

VOLUME 6E | CHAPTER 51

Chapter 51 - AWS WAF, Shield, Firewall Manager, and Network Firewall

Layered application, DDoS, and network traffic protection at organizational scale

Learning Objectives

- Differentiate AWS WAF, Shield Standard, Shield Advanced, Firewall Manager, and Network Firewall.
- Design rule evaluation, managed rules, rate controls, logging, central policy, and exception handling.
- Build layered edge and network protection without creating an untestable availability risk.

Why This Matters in Production

Public applications face volumetric attacks, application-layer abuse, automated bots, exploit attempts, and malicious network traffic. No single control covers every layer. AWS services can filter web requests, provide DDoS protections, centrally manage policies, and inspect VPC traffic.

Production principle Deploy defenses in layers, begin in count or alert mode where possible, test representative traffic, and treat firewall policy as versioned code with rollback.

Service Selection

Service	Layer and purpose	Typical placement
AWS WAF	Layer 7 HTTP/S request inspection and control	CloudFront, ALB, API Gateway, AppSync, and supported resources.
Shield Standard	Automatic baseline DDoS protection	Included for supported AWS resources.
Shield Advanced	Enhanced DDoS protection, visibility, and response features	Critical protected resources with business justification.
Firewall Manager	Central policy deployment across accounts	Security administrator account with Organizations.
Network Firewall	Stateful network traffic inspection and filtering	Centralized or distributed VPC inspection architecture.

WAF Rule Engineering

Web ACL capacity, rule priority, scope-down statements, labels, managed rule groups, custom rules, rate-based rules, and exception logic must be understood before enforcement. Start new managed rules in count mode, inspect sampled requests and logs, then promote with defined exclusions. Protect logs because they can include request data.

DDoS and Availability

DDoS preparation includes scalable architectures, CloudFront and Route 53 where appropriate, protected origin design, quotas, operational contacts, tested runbooks, and cost-protection understanding. Application-layer attacks still require WAF rules and capacity planning.

Network Firewall Architecture

Route tables must steer intended traffic through firewall endpoints symmetrically. Central inspection reduces policy duplication but increases routing complexity and shared-service dependency. Distributed inspection increases cost but may simplify failure boundaries. Validate fail-open/fail-closed expectations and maintenance behavior.

Production Design Checklist

- Protection requirements map to application, edge, and network layers.
- WAF changes begin with count/testing and use versioned policy.
- Logging, sampled requests, privacy, and retention are configured.
- Firewall Manager scope uses OUs, tags, and exceptions deliberately.
- Network Firewall routing is symmetric and tested for each AZ.
- DDoS escalation, contacts, quotas, and runbooks are rehearsed.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Legitimate requests blocked	Managed rule false positive or custom rule scope too broad	Inspect terminating rule, labels, and sampled request	Add narrow exclusion or scope-down statement; avoid disabling entire group.
Rule does not match	Wrong scope, priority, text transformation, or forwarded IP config	Test request and inspect logs	Correct statement and transformation; validate proxy headers safely.
Network traffic blackholed	Route asymmetry, endpoint/AZ mismatch, or firewall policy	Trace routes and flow logs	Correct AZ-aware routing and endpoint path.
Firewall Manager noncompliant	Account/OU/tag scope, prerequisite, or local resource conflict	Inspect policy compliance details	Correct scope and remediate or document approved exception.

Hands-on Lab 51: Deploy and Tune a Web ACL

Objective Protect a sandbox endpoint with managed and rate-based rules, analyze count-mode results, and promote a tested rule safely.

Implementation

1. Create a sandbox web endpoint behind a supported resource.
2. Create a web ACL with logging and a managed rule group in count mode.
3. Send normal and intentionally suspicious test requests.
4. Inspect sampled requests, labels, and logs.
5. Add a narrow rate-based rule and validate thresholds.
6. Promote one tested rule to block and verify normal traffic remains available.

Validation Evidence

- Suspicious requests are counted and later blocked by the intended rule.
- Normal transaction remains successful.
- Logs identify rule, action, request, and timestamp.

Cleanup

- Remove the training web ACL, log destination, and endpoint.
- Delete test data after the required evidence is captured.

Stretch challenge Manage the web ACL from Firewall Manager across two sandbox accounts with an OU-scoped exception process.

Chapter Summary

- AWS WAF protects HTTP/S requests; Shield addresses DDoS risk; Network Firewall inspects VPC traffic.
- Firewall Manager distributes policies across accounts and resources.
- Count-mode testing, logging, routing validation, and rollback protect availability.

Interview Questions and Model Answers

1. What is the difference between WAF and Network Firewall?

WAF evaluates web requests at Layer 7 for supported resources; Network Firewall inspects routed VPC network traffic with stateful/stateless rules.

2. Why use count mode?

It shows how a rule would match production traffic before enforcement, reducing false-positive risk.

3. What does Firewall Manager require?

AWS Organizations integration, a delegated administrator, and service-specific prerequisites for centralized policy management.

Official AWS References

WAF and Firewall Manager setup: <https://docs.aws.amazon.com/waf/latest/developerguide/setting-up-waf.html>

WAF policies: <https://docs.aws.amazon.com/waf/latest/developerguide/waf-policies.html>

Network Firewall policies: <https://docs.aws.amazon.com/waf/latest/developerguide/network-firewall-policies.html>

WAF or Shield decision guide: <https://docs.aws.amazon.com/pdfs/decision-guides/latest/waf-or-shield/waf-or-shield.pdf>

VOLUME 6E | CHAPTER 52

Chapter 52 - AWS Organizations, Control Tower, and Landing Zones

Multi-account boundaries, organizational policy, account vending, and scalable governance

Learning Objectives

- Design an AWS Organizations hierarchy with management, security, infrastructure, sandbox, and workload accounts.
- Explain SCPs, resource control policies, delegated administrators, tag policies, and backup policies.
- Use Control Tower landing zones, controls, Account Factory, and drift management.

Why This Matters in Production

An AWS account is a strong isolation and billing boundary. Enterprises need many accounts, but unmanaged account growth creates inconsistent identity, logging, security, and networking. Organizations and Control Tower provide the structure and governance needed to scale.

Production principle Use accounts to contain blast radius and ownership. Apply preventive policy carefully, test it in a staging OU, and keep the management account free of workloads.

Organization Design

Organizational units should reflect policy boundaries, not the current org chart alone. Common areas include Security, Infrastructure, Workloads, Sandbox, Suspended, Exceptions, and PolicyStaging. Separate production and nonproduction where policy or risk differs.

Account type	Purpose	Key controls
Management	Organizations billing and root governance	No workloads; tightly restricted access.
Log archive	Immutable organization audit storage	Restricted write/read, retention, integrity monitoring.
Security tooling	Delegated security administration and response	Central findings, investigation, automation.
Network/shared services	Transit, DNS, directory, tools	Controlled cross-account access and change.
Workload	Application resources by environment/team	SCPs, budgets, guardrails, owner lifecycle.

Policy Types

SCPs define maximum available permissions for principals in member accounts; they do not grant permissions. Resource control policies can limit permissions available to resources across the organization. Other policy types cover tags, backup, and service-specific governance. Always preserve documented emergency and service operation paths.

Control Tower Landing Zone

Control Tower orchestrates Organizations, IAM Identity Center, Service Catalog, CloudFormation StackSets, Config, CloudTrail, and other services to establish and govern a landing zone. Controls can be preventive, detective, or proactive depending on implementation. Account Factory standardizes provisioning.

Governance Rollout

Test controls in a PolicyStaging OU, inspect affected services and service-linked roles, deploy to a small OU, monitor denied actions, then expand. Track exceptions with owner, purpose, compensating control, and expiry. Repair landing-zone drift through supported Control Tower workflows.

Production Design Checklist

- Management account has no routine workloads and uses tightly controlled access.
- OU design maps to stable policy boundaries.
- Organization CloudTrail, Config, security services, and log archive are centralized.
- SCP and RCP changes are tested in PolicyStaging before broad rollout.
- Delegated administrators are assigned to dedicated platform/security accounts.
- Account provisioning, closure, suspension, ownership, and budget lifecycle are automated.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
SCP blocks required action	Policy denies action or removes required allow ceiling	Evaluate effective policies from account and OU path	Refine policy in staging; preserve least privilege and documented exception.
Control Tower account drift	Managed resource changed outside supported workflow	Review drift status and affected controls	Use landing-zone or account repair rather than manual patching.
New account lacks security service	Enrollment, delegated admin, Region, or lifecycle automation failed	Check account factory and organization integration	Repair enrollment and add lifecycle validation.
Service-linked role operation denied	Overly broad SCP/RCP affects AWS service	Inspect denied CloudTrail event and service docs	Add narrow policy exception using supported conditions.

Hands-on Lab 52: Design a Policy-Staged Multi-Account Landing Zone

Objective Create an OU and account governance design, test a preventive policy safely, and document account vending and exception workflows.

Implementation

1. Draw an organization hierarchy with management, security, infrastructure, workload, sandbox, and PolicyStaging OUs.
2. Define baseline identity, logging, Config, security, budgets, networking, and tagging requirements.
3. Write a small SCP that prevents a high-risk action while preserving approved administrative paths.
4. Apply it only to a sandbox account in PolicyStaging.
5. Test allowed and denied operations and review CloudTrail evidence.
6. Document promotion, rollback, exception, and break-glass procedures.

Validation Evidence

- Effective policy produces only the intended denial.
- CloudTrail records the denied action and principal.
- Rollback removes the policy impact without manual resource repair.

Cleanup

- Detach and delete the training policy if not required.
- Close or retain sandbox account according to organization process.

Stretch challenge Design an account-vending workflow that provisions budgets, identity assignments, baseline stacks, security service enrollment, and expiry automatically.

Chapter Summary

- Organizations provides multi-account structure and policy; Control Tower orchestrates a governed landing zone.
- SCPs limit maximum identity permissions and require careful staged testing.
- Account lifecycle, delegated administration, central logs, and policy exceptions must be operationalized.

Interview Questions and Model Answers

1. Do SCPs grant permissions?

No. They define the maximum permissions available; IAM and resource policies must still allow the action.

2. Why avoid workloads in the management account?

It is the highest-trust organization account; minimizing resources and access reduces blast radius and conflict.

3. What is a landing zone?

A governed multi-account foundation with identity, logging, security, networking, and account-provisioning controls.

Official AWS References

Organizations SCPs: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

Organizations RCPs: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_rcps.html

Control Tower overview: <https://docs.aws.amazon.com/controltower/latest/userguide/what-is-control-tower.html>

Multi-account strategy: <https://docs.aws.amazon.com/controltower/latest/userguide/aws-multi-account-landing-zone.html>

VOLUME 6E | CHAPTER 53

Chapter 53 - DevSecOps and Software Supply-Chain Security

Security controls from commit and dependency to artifact, deployment, and runtime

Learning Objectives

- Build a threat-informed DevSecOps control map for source, dependencies, build, artifact, deployment, and runtime.
- Use SBOMs, signing, provenance, scanning, policy as code, secret detection, and artifact immutability.
- Design exception, remediation, and developer feedback workflows that sustain delivery speed.

Why This Matters in Production

Modern applications depend on source repositories, package registries, base images, build services, pipeline plugins, and deployment roles. Attackers target this chain because one compromised component can affect many environments. Security must be integrated into the delivery system rather than added as a final review.

Production principle Establish trust at every transition: authenticated source, isolated build, verified dependency, immutable artifact, authorized promotion, and observable runtime.

Supply-Chain Threat Model

Stage	Threat	Control examples
Source	Stolen credentials, unreviewed changes, malicious commit	MFA/federation, branch protection, signed commits/tags, review, secret scanning.
Dependencies	Typosquatting, compromised package, vulnerable version	Lock files, approved registries, checksum verification, dependency scanning.
Build	Credential theft, tampered runner, poisoned cache	Ephemeral builds, minimal role, isolated network, pinned images, cache controls.
Artifact	Mutable tag, substitution, unauthorized access	Digest addressing, encryption, signing, provenance, immutable repository.
Deploy	Privilege escalation, policy bypass, environment drift	Cross-account roles, approvals, IaC policy checks, change sets, admission controls.
Runtime	Exploit, drift, exfiltration	Least privilege, segmentation, telemetry, detection, patching, response.

Security Gates and Feedback

Not every finding should block every pipeline. Define policy by severity, exploitability, asset class, internet exposure, and exception status. Fast checks run early; deeper integration and dynamic tests run later. Provide developers with actionable location, fix version, owner, and policy rationale.

Artifact Trust

Produce an SBOM and provenance metadata during the build. Sign artifacts or images with approved keys or identities. Verify signature and digest at promotion or admission. Store evidence in a tamper-resistant location. Mutable tags may support human readability but must resolve to immutable digests for deployment.

Policy as Code

Evaluate CloudFormation/CDK output, IAM policies, Kubernetes manifests, container configuration, and organization controls before deployment. Policy repositories require versioning, tests, staged rollout, ownership, exception expiry, and metrics for false positives and bypass attempts.

Production Design Checklist

- Source protection and review requirements are enforced.
- Dependencies, base images, and build environments are pinned and scanned.
- Build roles are minimal, ephemeral, and separated by environment.
- Artifacts have digest, SBOM, provenance, scan results, and signature evidence.
- Deployment verifies artifact identity and policy compliance.
- Exceptions have risk owner, compensating controls, and expiry.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Pipeline blocks too many releases	Policy ignores context or scanner produces noisy findings	Analyze false-positive and exception data	Tune risk policy, improve rule specificity, and keep critical controls mandatory.
Signed artifact fails verification	Wrong key/identity, digest changed, or metadata mismatch	Compare digest, signature subject, and trust policy	Reject artifact and rebuild through trusted pipeline.
Secret appears in history	Secret detection occurred after commit	Revoke immediately, inspect use, purge where appropriate	Rotate secret, investigate, add pre-commit and server-side prevention.
SBOM incomplete	Build tool misses vendored, generated, or OS packages	Compare runtime/image inventory	Combine ecosystem and image scanning and validate coverage.

Hands-on Lab 53: Create a Trusted Artifact Pipeline

Objective Produce an application artifact with tests, SBOM, vulnerability report, digest, signature, provenance record, and deployment verification.

Implementation

1. Protect the source branch and require reviewed changes.
2. Build in CodeBuild with pinned dependencies and a minimal service role.
3. Generate an SBOM and scan dependencies or image packages.
4. Hash and sign the artifact or image.
5. Store artifact and evidence immutably.
6. Verify signature, digest, and policy in a deployment stage before promotion.

Validation Evidence

- Artifact digest matches the signed value.
- SBOM and scan report correspond to the same build identity.
- A modified artifact is rejected during verification.

Cleanup

- Delete sandbox artifacts and signing resources only according to key-retention policy.
- Remove temporary pipeline and roles.

Stretch challenge Add an exception workflow that requires security approval, records compensating controls, and automatically expires the exception.

Chapter Summary

- DevSecOps secures the complete delivery chain, not only application code.
- Artifact identity, provenance, SBOMs, signing, and policy checks support trustworthy promotion.
- Risk-based gates and high-quality developer feedback maintain both safety and flow.

Interview Questions and Model Answers

1. What is an SBOM?

A machine-readable inventory of software components and dependencies associated with an artifact.

2. Why sign an artifact?

A signature allows consumers to verify artifact identity and that it was approved by a trusted signer or build identity.

3. Should every vulnerability block deployment?

Policy should consider severity, exploitability, exposure, asset criticality, fix availability, and approved time-bound exceptions.

Official AWS References

CDK best practices: <https://docs.aws.amazon.com/cdk/v2/guide/best-practices.html>

Security Hub control reference: <https://docs.aws.amazon.com/securityhub/latest/userguide/securityhub-controls-reference.html>

Amazon Inspector documentation: <https://docs.aws.amazon.com/inspector/>

AWS Signer documentation: <https://docs.aws.amazon.com/signer/>

VOLUME 6E | CHAPTER 54

Chapter 54 - Well-Architected, Resilience Engineering, and FinOps

Architecture review, recovery objectives, failure testing, and sustainable cloud economics

Learning Objectives

- Apply the six Well-Architected pillars and workload lenses to design decisions.
- Define RTO, RPO, dependency recovery, resilience policies, and game-day validation.
- Implement cost allocation, budgets, optimization, unit economics, and engineering ownership.

Why This Matters in Production

A system can be secure but unreliable, reliable but unaffordable, or efficient but operationally opaque. Well-Architected provides a structured review model. Resilience engineering turns availability goals into tested recovery capability. FinOps makes cost a shared engineering and business signal.

Production principle Architecture quality is demonstrated through measurable outcomes: recovery tests, operational evidence, security posture, performance, sustainability, and cost per business unit.

Well-Architected Review

The pillars are Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. Reviews should be blameless and evidence-driven. Record risks, owners, due dates, and tradeoffs; do not treat the review as a certification or one-time checklist.

Resilience Engineering

Define application components, dependencies, failure modes, RTO, RPO, availability targets, and data integrity requirements. Choose strategies such as backup and restore, pilot light, warm standby, or multi-site based on business need. AWS Resilience Hub can assess supported applications against resilience policies and recommend improvements.

Objective	Meaning	Validation
RTO	Maximum acceptable time to restore service	Timed failover and recovery exercise.
RPO	Maximum acceptable data loss measured in time	Restore or replica-lag evidence.
Availability SLO	Target successful service over a window	SLI and error-budget calculation.
Dependency tolerance	Behavior when downstream service fails	Fault injection, timeout, retry, circuit-breaker test.

FinOps Operating Model

Establish cost allocation tags and account structure, budgets, anomaly detection, forecasting, commitment management, rightsizing, storage lifecycle, idle-resource cleanup, and unit-cost measures. Engineering teams should see cost alongside reliability and performance. Avoid optimizing only the bill while increasing labor or risk.

Game Days and Continuous Improvement

A game day tests a defined hypothesis under controlled conditions. Participants observe detection, escalation, diagnosis, mitigation, communication, and recovery. Collect timing and evidence, then convert findings into prioritized engineering work. Never perform uncontrolled destructive testing in production.

Production Design Checklist

- Well-Architected risks have owners, due dates, and accepted rationale.
- RTO, RPO, SLOs, and dependency assumptions are documented.
- Backups are restored regularly, not merely reported as successful.
- Failover and failback procedures are automated and tested.
- Costs are allocated to owners and measured with business unit metrics.
- Commitments and optimizations consider forecast, architecture, and flexibility.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Backup succeeds but restore fails	Missing permissions, dependencies, encryption key, or incomplete procedure	Run full restore test and inspect logs	Fix restore runbook, key access, dependencies, and validation.
Failover exceeds RTO	Manual steps, DNS TTL, data recovery, or capacity delay	Time each recovery phase	Automate bottlenecks and pre-provision justified capacity.
Cost rises without traffic growth	Idle resources, telemetry, data transfer, pricing change, or inefficient workload	Analyze cost by service/account/tag and usage	Assign owner and remove waste or redesign unit economics.
Well-Architected actions never close	No owner, priority, budget, or governance	Review risk register and leadership cadence	Integrate actions into backlog and objective tracking.

Hands-on Lab 54: Run a Resilience and Cost Review

Objective Define recovery objectives for a sandbox workload, test one failure scenario, measure recovery, and calculate a basic unit cost.

Implementation

1. Document architecture, dependencies, RTO, RPO, availability target, and critical user journey.
2. Perform a focused Well-Architected review and record top risks.
3. Configure or model a Resilience Hub policy where supported.
4. Inject a controlled dependency or instance failure.
5. Measure detection, mitigation, recovery, and data state.
6. Calculate monthly cost and cost per transaction; identify one safe optimization.

Validation Evidence

- Measured recovery time is compared with RTO.
- Data recovery evidence is compared with RPO.
- Risk and cost actions have owners and target dates.

Cleanup

- Remove temporary fault-injection and test resources.
- Return sandbox capacity to normal and confirm service health.

Stretch challenge Perform a second game day after remediation and demonstrate measurable improvement in recovery time or error-budget impact.

Chapter Summary

- Well-Architected reviews expose cross-pillar risks and tradeoffs.
- Resilience requires tested recovery objectives, not assumptions.
- FinOps integrates cost allocation, optimization, forecasting, and unit economics into engineering work.

Interview Questions and Model Answers

1. What is the difference between RTO and RPO?

RTO is acceptable restoration time; RPO is acceptable data loss measured as time.

2. What is an error budget?

The allowable unreliability derived from an SLO over a defined window.

3. Why restore-test backups?

A successful backup job does not prove data, permissions, dependencies, keys, and procedures can restore a working service.

Official AWS References

AWS Resilience Hub: <https://docs.aws.amazon.com/resilience-hub/latest/userguide/what-is.html>

Resilience recommendations: <https://docs.aws.amazon.com/resilience-hub/latest/userguide/resil-recs.html>

Cost Optimization Pillar: <https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html>

Well-Architected cost optimization: <https://docs.aws.amazon.com/wellarchitected/latest/framework/cost-optimization.html>

VOLUME 6E | CHAPTER 55

Chapter 55 - Enterprise Landing Zone and Secure Delivery Capstone

Integrating governance, IaC, CI/CD, observability, security operations, resilience, and FinOps

Capstone Mission

Design and implement a production-oriented AWS operating foundation for a regulated digital commerce company. The platform must provision accounts safely, deliver infrastructure and applications through reviewed pipelines, centralize evidence, detect threats, protect public endpoints, support incident response, test recovery, and allocate cost to accountable teams.

Business Scenario

- The company operates development, test, staging, and production workloads across multiple teams.
- Customer-facing APIs require strong availability and protection from abuse.
- Audit evidence must be centralized and protected from workload administrators.
- Production changes require separation of duties, artifact provenance, approval, health verification, and rollback.
- Security findings require central triage and measurable remediation targets.
- Leadership requires cost allocation by product, environment, and owner.

Target Account Structure

OU or account	Responsibilities	Critical controls
Management account	Organizations and billing only	No workloads, root MFA, restricted access.
Security OU: Log Archive	CloudTrail, Config snapshots, WAF/security logs	Immutable storage, KMS, limited reader role.
Security OU: Security Tooling	GuardDuty, Inspector, Macie, Security Hub, Detective	Delegated admin, EventBridge triage automation.
Infrastructure OU: Network	Transit, DNS, inspection, endpoints	Change-controlled routing and firewall policy.
Infrastructure OU: Tooling	Pipelines, artifacts, construct libraries	Cross-account deployment roles and artifact signing.
Workloads OUs	Dev/Test and Prod accounts per product boundary	SCPs, budgets, identity, baseline stacks, owner tags.
Sandbox OU	Time-limited experimentation	Budget caps, restricted services, automated expiry.

Reference Architecture Flow

```
Developer pull request
-> review + source controls
-> CodePipeline in Tooling account
-> CodeBuild tests + SBOM + scan + signed immutable artifact
-> CloudFormation/CDK change set + policy checks
-> deploy to staging account
```

```

-> CloudWatch/Application Signals verification
-> approval with evidence
-> assume production deployment role
-> gradual rollout + automatic rollback alarms
-> CloudTrail/Config/Security Hub evidence in security accounts

```

Mandatory Workstreams

Workstream	Deliverables	Acceptance criteria
Landing zone	OU design, account vending, identity assignments, SCPs/RCPs, central logs	New sandbox account receives baseline within defined target and passes control checks.
Infrastructure as Code	CDK constructs or CloudFormation modules, tests, change-set review	No manual production resources; drift monitoring and protected stateful resources.
Delivery platform	Cross-account pipeline, immutable artifact, signing, approvals, rollback	Same digest promoted; failed health test causes automatic rollback.
Observability	Structured logs, metrics, traces, SLOs, dashboards, synthetics	Critical transaction failure detected and routed within target time.
Security operations	Organization services, finding aggregation, triage automation	Sample high-severity finding creates owned case with evidence.
Perimeter defense	WAF managed/custom rules, rate limiting, logging, DDoS runbook	Malicious test blocked; normal traffic remains healthy.
Operations	Session Manager, patch compliance, automation runbooks	No inbound SSH/RDP; canary patch process completes with evidence.
Resilience/FinOps	RTO/RPO tests, restore/failover, budgets, unit cost	Recovery meets target or creates tracked remediation; costs allocated to owner.

Implementation Phases

1. Create architecture decision records, data classifications, service ownership, SLOs, RTO/RPO, and threat model.
2. Build the organization and landing-zone design in a lab organization or simulate it with documented policies where account creation is unavailable.
3. Establish central logging, Config, and security delegated administration.
4. Create governed CloudFormation/CDK modules and policy tests.
5. Build the cross-account release pipeline with artifact evidence and a staging environment.
6. Instrument the application with metrics, logs, traces, SLOs, and synthetics.
7. Apply WAF protections and a security finding triage workflow.
8. Run a failed deployment test, security sample finding, dependency outage, and restore test.
9. Measure recovery, alerting, pipeline, security response, and cost outcomes.
10. Present a production-readiness review with residual risks and prioritized remediation.

Required Failure Tests

Test	Expected behavior	Evidence
Unsafe IaC replacement	Policy or change-set review blocks approval	Change set, policy finding, reviewer decision.
Unhealthy application release	Alarm or lifecycle validation rolls back	Deployment events, metric graph, restored version.
Stolen or unauthorized role attempt	IAM/SCP denies and security event routes	CloudTrail denial, finding or alert, case record.
WAF abuse burst	Rate rule limits malicious client without broad outage	WAF log and healthy synthetic transaction.
Managed-node patch failure	Rate control stops cohort and preserves healthy capacity	SSM command status and incident note.
Regional or dependency failure simulation	Runbook activates and service recovers within measured target	Timeline, trace/metrics, data validation.
Budget threshold breach	Owner receives actionable alert and investigates driver	Budget alert, cost analysis, remediation action.

Security and Governance Requirements

- All human access uses IAM Identity Center or federation with MFA and temporary credentials.
- Production deployment is performed by pipeline roles; developer roles cannot deploy directly.
- KMS keys separate administration from usage and protect artifacts, logs, secrets, and backups.
- SCPs are tested in PolicyStaging and preserve approved break-glass and service-linked behavior.
- Audit and security evidence is retained in dedicated accounts with restricted deletion.
- Secrets are retrieved at runtime and rotated; no plaintext appears in source, build output, images, or logs.
- Exceptions require risk owner, rationale, compensating control, approval, and expiry.

Evidence Package

- Architecture diagram and account/OU policy map.
- IaC repository, tests, synthesized templates, change sets, and drift report.
- Pipeline execution with artifact digest, SBOM, scan, signature, approvals, and rollback test.
- CloudWatch dashboard, SLO, alarm history, trace, and synthetic evidence.
- CloudTrail event, Config timeline, sample security finding, triage record, and remediation action.
- WAF logs and Network/edge protection test.
- Systems Manager session, patch compliance, and automation output.
- Recovery test timeline, RTO/RPO comparison, monthly cost allocation, and unit-cost calculation.

Capstone Scoring Rubric

Area	Weight	Excellent evidence
Security and governance	20%	Least privilege, separation of duties, central evidence, staged policy, tested exceptions.
Automation and repeatability	20%	Full IaC and pipeline delivery with deterministic artifacts and recovery.
Reliability and observability	20%	SLOs, actionable alarms, traces, tested

Area	Weight	Excellent evidence
		rollback and recovery.
Operational readiness	15%	Runbooks, ownership, Session Manager, patching, incident workflow.
Supply-chain integrity	10%	SBOM, signing, digest promotion, policy gates, protected source.
Cost and sustainability	10%	Allocation, budgets, unit cost, justified optimization.
Documentation and presentation	5%	Clear decisions, diagrams, evidence, residual risks, and roadmap.

Final Interview Panel

Interview Questions and Model Answers

1. How does your design prevent a developer laptop from becoming a production control plane?

Production deployment occurs through centrally governed pipelines and cross-account roles. Developer identities can propose changes but cannot execute direct production changes.

2. How do you prove that the artifact tested in staging is the artifact running in production?

The pipeline records and verifies a single immutable digest and signature across both environments.

3. How do you avoid an SCP causing an organization-wide outage?

Test in PolicyStaging, evaluate service-linked behavior, deploy progressively, monitor denials, preserve rollback and break-glass paths.

4. How is a critical security finding handled?

It is aggregated, enriched with business context, assigned an owner, optionally contained through reversible automation, investigated, remediated, and closed with evidence.

5. What is your most important residual risk?

The candidate should identify a real architecture-specific risk, quantify impact and likelihood, describe compensating controls, and provide a prioritized remediation plan.

Capstone Completion Checklist

- Architecture and ownership approved.
- Account, identity, audit, security, network, and budget baselines implemented.
- Infrastructure and application changes delivered through reviewed pipelines.
- Observability detects user-impacting failures and supports trace-to-log correlation.
- Security findings and perimeter events enter an owned response workflow.
- Rollback, restore, and failover tests have measured outcomes.
- Costs are allocated and one safe optimization is verified.
- Residual risks and follow-up actions are documented.

Official AWS References

AWS Control Tower: <https://docs.aws.amazon.com/controltower/>

CloudFormation: <https://docs.aws.amazon.com/cloudformation/>

CodePipeline: <https://docs.aws.amazon.com/codepipeline/>

CloudWatch: <https://docs.aws.amazon.com/cloudwatch/>

Security Hub: <https://docs.aws.amazon.com/securityhub/>

Well-Architected Framework: <https://docs.aws.amazon.com/wellarchitected/>

Part E Command Reference

Task	Command
CloudFormation validate	<code>aws cloudformation validate-template --template-body file://template.yaml</code>
Create change set	<code>aws cloudformation create-change-set --stack-name NAME --change-set-name CHANGE --change-set-type UPDATE --template-body file://template.yaml --capabilities CAPABILITY_NAMED_IAM</code>
CDK preview	<code>cdk synth && cdk diff</code>
Pipeline state	<code>aws codepipeline get-pipeline-state --name PIPELINE</code>
Build details	<code>aws codebuild batch-get-builds --ids BUILD_ID</code>
Deployment status	<code>aws deploy get-deployment --deployment-id DEPLOYMENT_ID</code>
Alarm history	<code>aws cloudwatch describe-alarm-history --alarm-name NAME</code>
Trace summaries	<code>aws xray get-trace-summaries --start-time TIME --end-time TIME</code>
Trail status	<code>aws cloudtrail get-trail-status --name TRAIL</code>
Config status	<code>aws configservice describe-configuration-recorder-status</code>
Managed nodes	<code>aws ssm describe-instance-information</code>
Start session	<code>aws ssm start-session --target INSTANCE_ID</code>
Read secret	<code>aws secretsmanager get-secret-value --secret-id SECRET</code>
Security findings	<code>aws securityhub get-findings --filters file://filters.json</code>
Organization policies	<code>aws organizations list-policies --filter SERVICE_CONTROL_POLICY</code>

Cross-Service Troubleshooting Sequence

1. Confirm current account, role, Region, and organization context.
2. Identify the first failed control-plane or data-plane operation from CloudTrail, pipeline, or service events.
3. Check IAM, resource policy, KMS policy, SCP/RCP, endpoint policy, and session context.
4. Inspect deployment, stack, build, command, alarm, trace, or finding history in time order.
5. Compare desired IaC state with actual configuration and drift.
6. Validate network path, DNS, routes, security controls, proxies, and service endpoints.
7. Check quotas, throttling, service health, capacity, and billing state.
8. Apply the smallest reversible correction and preserve evidence.
9. Update automation, tests, policy, runbook, and monitoring to prevent recurrence.

Production Readiness Checklist

- All infrastructure is versioned, reviewed, policy-checked, and deployed through controlled roles.
- Pipelines promote immutable, signed artifacts and support health-based rollback.
- Metrics, logs, traces, SLOs, synthetics, alarms, dashboards, owners, and runbooks are complete.
- Audit evidence is centralized, encrypted, retained, and protected from workload administrators.

- Fleet administration uses Systems Manager and automated patch/change cohorts.
- Secrets, keys, parameters, and certificates have lifecycle ownership and rotation monitoring.
- Threat, vulnerability, sensitive-data, and posture services cover required accounts and Regions.
- Edge and network controls are logged, tested, centrally governed, and reversible.
- Organization policies and Control Tower controls are staged and exception-managed.
- Resilience objectives are tested and costs are allocated to accountable teams.

Glossary

Term	Definition
Change set	A preview of proposed CloudFormation stack changes before execution.
Construct	A CDK building block representing one or more cloud components.
Artifact provenance	Evidence describing how, where, and from which inputs an artifact was produced.
SLI / SLO	A measured service indicator and its target objective over a defined window.
Trace / span	An end-to-end transaction and a timed unit of work within it.
Configuration item	AWS Config record of a supported resource state and relationships.
Managed node	A machine registered and communicating with AWS Systems Manager.
Envelope encryption	Encrypting data with a data key protected by a higher-level KMS key.
Delegated administrator	A member account authorized to administer an AWS service for an organization.
SCP	Organization policy that limits maximum permissions in member accounts.
Landing zone	A governed multi-account AWS foundation.
SBOM	Software bill of materials listing components associated with an artifact.
RTO / RPO	Maximum acceptable recovery time and data loss measured in time.
FinOps	Collaborative practice for cloud financial accountability and value optimization.

Official Documentation Index

AWS CloudFormation: <https://docs.aws.amazon.com/cloudformation/>

AWS CDK: <https://docs.aws.amazon.com/cdk/>

AWS CodePipeline: <https://docs.aws.amazon.com/codepipeline/>

AWS CodeBuild: <https://docs.aws.amazon.com/codebuild/>

AWS CodeDeploy: <https://docs.aws.amazon.com/codedeploy/>

Amazon CloudWatch: <https://docs.aws.amazon.com/cloudwatch/>

AWS X-Ray: <https://docs.aws.amazon.com/xray/>

AWS CloudTrail: <https://docs.aws.amazon.com/cloudtrail/>

AWS Config: <https://docs.aws.amazon.com/config/>

AWS Systems Manager: <https://docs.aws.amazon.com/systems-manager/>

AWS KMS: <https://docs.aws.amazon.com/kms/>

AWS Secrets Manager: <https://docs.aws.amazon.com/secretsmanager/>

AWS Security Hub: <https://docs.aws.amazon.com/securityhub/>

Amazon GuardDuty: <https://docs.aws.amazon.com/guardduty/>

Amazon Inspector: <https://docs.aws.amazon.com/inspector/>

Amazon Macie: <https://docs.aws.amazon.com/macie/>

AWS WAF and Firewall Manager: <https://docs.aws.amazon.com/waf/>

AWS Organizations: <https://docs.aws.amazon.com/organizations/>

AWS Control Tower: <https://docs.aws.amazon.com/controltower/>

AWS Resilience Hub: <https://docs.aws.amazon.com/resilience-hub/>

AWS Well-Architected: <https://docs.aws.amazon.com/wellarchitected/>

End of Volume 6E You are ready for Volume 6F: enterprise projects, advanced architecture scenarios, certification and interview mastery, and the final integrated AWS engineering portfolio.